iRAYPLE Industrial Camera

SDK Development Document (Python)



V1.1.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	Jul. 2023	V1.0.0	First Release	V2.3.1
2	Mar. 2024	V1.1.0	Added matched routines of multi-camera and image-saving interface.	V2.4.2

Table of Contents

1 Overview 4
1.1 Compatibility Introduction 4
1.2 Installation Directory 4
1.2.1 SDK for Windows 4
1.2.2 SDK for Linux 6
1.3 Related Resources of SDK 6
1.4 SDK Engineer Development Configuration 6
2 Overall Call Process of SDK 8
3 Function Interface Calling of SDK 9
3.1 System Operation Instruction 9
3.1.1 Obtain Version Information 9
3.1.2 Enumeration Device (Find Device) 9
3.1.3 Handle Creation and Destruction of Device 10
3.2 Camera Operation Instruction 11
3.2.1 Camera Open and Close 11
3.2.2 Camera Configuration Download, Storage and Loading 12
3.2.3 Camera Configuration Settings (Only for GigE Camera) 13
3.2.4 Unique Features 14
3.3 Register Event Callback 15
3.4 Camera Stream Resource Operation Instruction 17
3.4.1 Stream Data Configuration 17
3.4.2 Image Collection 18
3.4.3 Acquire ImageData 20
3.4.4 Operations on Streaming and Image Data 21
3.5 Attribute Operation Instruction 22
3.5.1 Determination, Acquisition and Setting of Attribute 22
3.5.2 Execution of Command Attribute 28
3.5.3 Attribute Search and Usage 28
3.6 Image Operation 36
3.6.1 Video Recording 36
3.6.2 Image Operation 36
4 Matched Routine Description 39
5 FAQ (Frequently Asked Question) 40
5.1 Only 8 Images are Obtained by Using the Python Interface 40

1 Overview

1.1 Compatibility Introduction

Table 1-1 Operating system requirement

Item	Version	Note
Windows *1	Win7 SP1 32/64bit	Ensure that OS version includes the patch of SP1.
	Win8 32/64bit	
	Win10 32/64bit	
Linux (x86) *2	System version: Ubuntu 12.04 LTS~Ubuntu 21.04 LTS (32/64bit)	The upgraded kernel version system is excluded.
	CentOS 6.5~CentOS 8.0 (32/64bit)	The upgraded kernel version system is excluded.

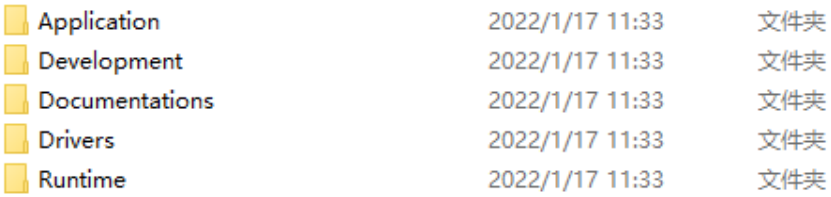
- Windows XP is no longer supported since SDK V2.2.5.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).
- Use the command of **uname–a** to check the kernel version number.

1.2 Installation Directory

1.2.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-1 Installation directory structure



In V2.1.0, user can select the installation path.

Table 1-2 Installation directory structure description

Application	The client software is compatible with 32-bit and 64-bit. The matched tool is compatible with 32-bit and 64-bit.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm platform (Halcon and Sherlock).
	DOTENT DLL It includes .Net library file which is imported when user create the project.
	Include It includes the header file of the SDK interface, which explains the basic attributes of various interfaces. The specific usage of header files is in the samples folder. The other is the extension definition of the header file of the SDK, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.
	Samples It includes the various types of demo file, such as C#, Delphi, Lab View, MFC, Python, QT, VB and VC.
Documentations	**.Net** interface document **ActiveX** interface document **C** interface document SDK development document
Drivers	The installation and uninstallation software of DShowFilter. The installation and uninstallation software of GigE high-performance driver. The installation and uninstallation software of PCIex86 smart camera driver. The installation and uninstallation software of USB high-performance driver.
Runtime	The compilation dynamic link library when running the SDK. It includes the third-party GenICam library which is separated into

	two folders.

1.2.2 SDK for Linux

The installation directory of SDK for Linux is **/opt/Company Name/product name**.

The folder structure and introduction are as follows:

Figure 1-2 folder structure description

```
MVviewer
  ├ bin         : GUI Tool, discover/connect camera, image acquisition, properties setting, uninstall script and etc.;
  ├ include     : SDK head files, image convert head files;
  ├ lib         : SDK dll, third-party library, image converting lib;
  ├ module
  |  ├ GigEDriver : GigE camera driver;
  |  └ USBDriver  : USB camera driver;
  └ share
     ├ C        : C samples;
     ├ Qt       : Qt samples;
     └ Python   : Python samples;
```

1.3 Related Resources of SDK

Figure 1-3 SDK source description

Category	Item	File Storage Folder
SDK Library	Header File	【SDK Installation Directory】 \Development\Samples\Python\IMV\IMVApi.py \Development\Samples\Python\IMV\IMVDefines.py
	Runtime Library (32-bit)	【SDK Installation Directory】 \Runtime \Win32\MVSDKmd.dll
	Runtime Library (64-bit)	【SDK Installation Directory】 \Runtime \x64\MVSDKmd.dll
Python Routine	—	【SDK Installation Directory】 \Development\Sample\Python\IMV

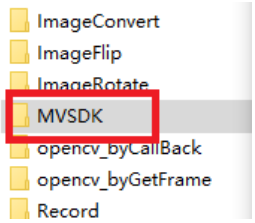
1.4 SDK Engineer Development Configuration

Take x64 as an example to demonstrate the method of engineer configuration.

Copy the header file

【SDK Installation Directory】\Development\Sample\Python\IMV

Figure 1-4 Header file



Import the header file module

Figure 1-5 Import the header file module

```
sys.path.append("../MVSDK")
from IMVApi import *
```

Import the dynamic library

User can change the path loading the dynamic library in the header file of the IMVApi.py to the absolute path of MVSDKmd.dll.

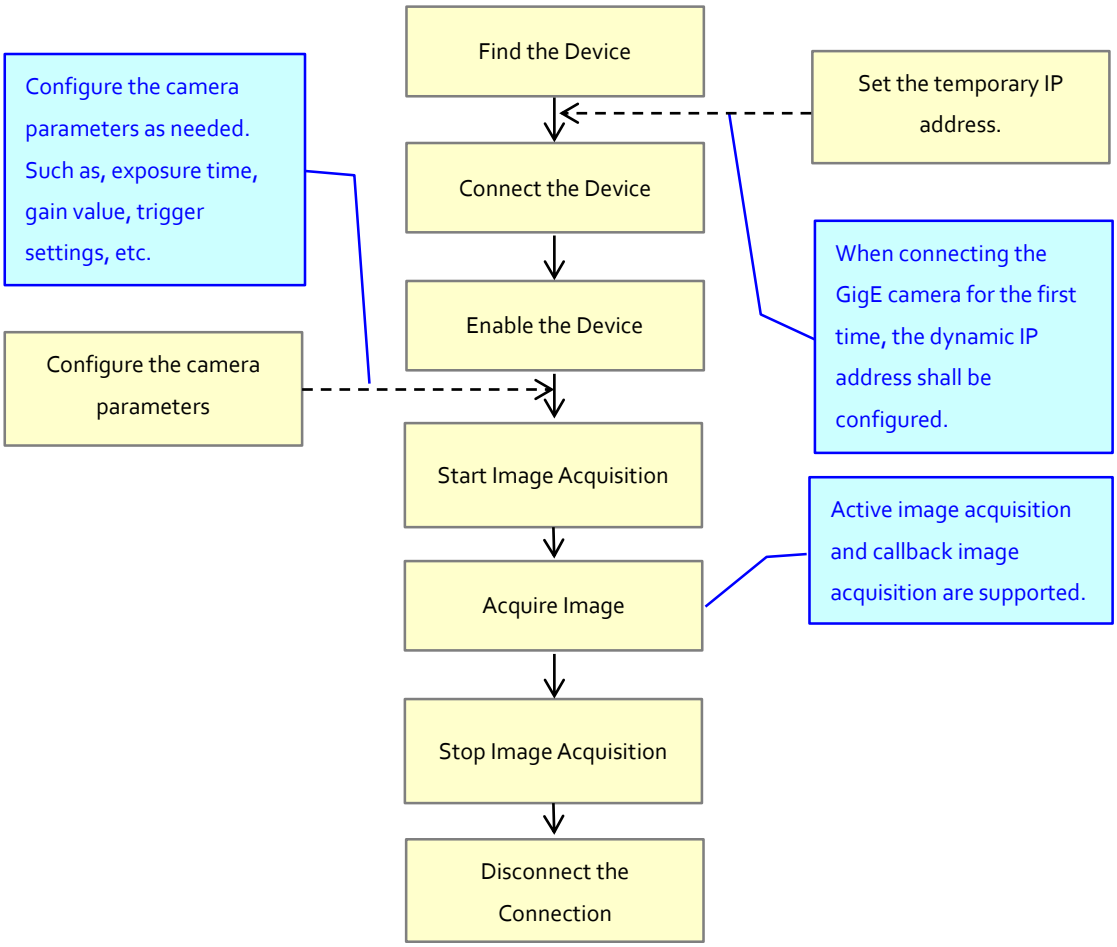
【SDK Installation Directory】 \Runtime\x64\MVSDKmd.dll

Figure 1-6 Dynamic library interface

```
# load SDK library
# 32bit
#MVSDKdll = WinDLL("../../../../../Runtime/Win32/MVSDKmd.dll")
# 64bit
MVSDKdll = WinDLL("../../../../../Runtime/x64/MVSDKmd.dll")
```

2 Overall Call Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of SDK

The directory of interface header files of SDK:

\Development\Samples\Python\IMV\MVSDK\IMVApi.py

The directory of the macro definition, enumeration, struct and callback function statement of SDK interface: **\Development\Samples\Python\IMV\ \MVSDK\IMVDefines.py**

Object description: **self** refers to the instance objects of the **MvCamera** being used.

3.1 System Operation Instruction

3.1.1 Obtain Version Information

- Interface: **IMV_GetVersion()**
  - ◇ Description: Obtain the OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: None.

3.1.2 Device Enumeration (Find Device)

- Interface: **IMV_EnumDevices (pDeviceList, interfaceType)**

Table 3-1 Parameter description

Parameter	I/O	Type	Description
pDeviceList	OUT	IMV_DeviceList	Device list.
interfaceType	IN	c_uint	The interface type to be enumerated can be combined arbitrarily, such as **interfaceTypeGige | interfaceTypeUsb3**.

  - ◇ Description: Enumeration Device. If succeeded, return **IMV_OK**. If failed, return error code.
    - □ When **interfaceType** = **interfaceTypeAll**, enumerate all online devices connected through different interfaces.
    - □ When **interfaceType** = **interfaceTypeGige**, enumerate all online devices connected through the GigE interface.
    - □ When **interfaceType** = **interfaceTypeUsb3**, enumerate all online devices connected through the USB interface.

- When **interfaceType** = **interfaceTypeCL**, enumerate the online devices connected through the CameraLink interface.
- The interfaceType of the **IMV_EnumDevices** supports any combination of interface types. For example, if you need to enumerate all online devices connected through the GigE port and USB 3.0 port, the **interfaceType** can be written as the **interfaceType = interfaceTypeGige | interfaceTypeUsb3**.

◇ Reference Routine: All current routines can be referenced.

- Interface: **IMV_EnumDevicesByUnicast(pDeviceList, pIpAddress)**

Table 3-2 Parameter description

Parameter	I/O	Type	Description
pDeviceList	OUT	IMV_DeviceList	Device list.
pIpAddress	IN	String	The IP address of the device.

◇ Description: Enumerate devices in unicast format, only for GigE model device. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

3.1.3 Handle Creation and Destruction of Device

- Interface: **IMV_CreateHandle (self, mode, pIdentifier)**

Table 3-3 Parameter description

Parameter	I/O	Type	Description
mode	IN	IMV_ECreateHandleMode	The handle creation mode of device.
pIdentifier	IN	bytes	Specify the identifier (device key, custom device name, explicitly casting the pointer representing the IP address from **char** type to **void** type; Explicitly casting the pointer representing the index from **unsigned int** type to **void** type).

◇ Description: Create the handle by specifying the identifier, such as the specified index, device key, custom device name and IP address. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

- Interface: **IMV_DestroyHandle(self)**

◇ Description: Destroy the handle of device. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

3.2 Camera Operation Instruction

- Interface: **IMV_GetDeviceInfo (self, pDevInfo)**

Table 3-4 Parameter description

Parameter	I/O	Type	Description
pDevInfo	OUT	IMV_DeviceInfo	Device information.

◇ Description: Obtain device information. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

3.2.1 Camera Open and Close

- Interface: **IMV_Open(self)**

◇ Description: Open the device (connect the device according to the handle). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: All current routines can be referenced.

- Interface: **IMV_OpenEx(self, accessPermission)**

Table 3-5 Parameter description

Parameter	I/O	Type	Description
accessPermission	IN	IMV_ECameraAccessPermission	Control the channel permission (the **accessPermissionControl** is adopted by default in IMV_Open).

◇ Description: Open the device (connect the device according to the handle and access permission). If succeeded, return **IMV_OK**. If failed, return error code. When using the **IMV_OpenEx** to execute the connection operation, the access rights will be determined.

◇ Reference Routine: None.

- Interface: **IMV_IsOpen(self)**

◇ Description: Determine whether the device is opened. If opened, return True. If close or offline, return False.

◇ Reference Routine: None.

- Interface: **IMV_Close(self)**
  - ◇ Description: Close Device (Close device according to the handle). If succeeded, return IMV_OK. If failed, return error code.
  - ◇ Reference Routine: All current routines can be referenced.

3.2.2 Camera Configuration Download, Storage and Loading

- Interface: **IMV_DownLoadGenICamXML(self, pFullFileName)**

Table 3-6 Parameter description

Parameter	I/O	Type	Description
pFullFileName	IN	str	File saving path.

◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_SaveDeviceCfg(self, pFullFileName)**

Table 3-7 Parameter description

Parameter	I/O	Type	Description
pFullFileName	IN	str	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.

◇ Description: Save the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_LoadDeviceCfg(self, pFullFileName, pErrorList)**

Table 3-8 Parameter description

Parameter	I/O	Type	Description
pFullFileName	IN	str	Full name (including saving path) of the exported device configuration file such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList	OUT	IMV_ErrorList	The list of attribute name that failed to load. The upper limit of attributes that store load failures is

			IMV_MAX_ERROR_LIST_NUM.

◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

3.2.3 Camera Configuration Settings (Only for GigE Camera)

- Interface: **IMV_GIGE_ForceIpAddress(self, pIpAddress, pSubnetMask, pGateway)**

Table 3-9 Parameter description

Parameter	I/O	Type	Description
pIpAddress	IN	str	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.
pSubnetMask	IN	str	The list of attribute name that failed to load. The upper limit of attributes that store load failures is **IMV_MAX_ERROR_LIST_NUM**.
pGateway	IN	str	Default gateway.

◇ Description: Modify the IP address of device (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

□ If both **pSubnetMask** and **pGateway** are configured with valid value when calling this function, the valid values shall prevail.

□ If both **pSubnetMask** and **pGateway** are configured with **NULL** when calling this function, replace it with subnet mask and gateway of the connected network card during the internal implementation.

□ If one of the **pSubnetMask** and **pGateway** is configured with NULL and the other is **not NULL**, an error code will be returned.

◇ Reference Routine: None.

- Interface: **IMV_GIGE_GetAccessPermission(self, pAccessPermission)**

Table 3-10 Parameter description

Parameter	I/O	Type	Description
pAccessPermission	OUT	IMV_ECameraAccessPermission	Current access rights of the device.

◇ Description: Obtain the current access rights of the device (only for GigE model device). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GIGE_SetAnswerTimeout(self, timeout)**

Table 3-11 Parameter description

Parameter	I/O	Type	Description
timeout	IN	c_uint	Timeout period (unit: s).

◇ Description: Configure the timeout period for device's response to SDK command (only for GigE model device). If succeeded, return IMV_OK. If failed, return error code.

◇ Reference Routine: None.

3.2.4 Unique Features

- Interface: **IMV_WriteUserPrivateData(self, pBuffer, pLength)**

Table 3-12 Parameter description

Parameter	I/O	Type	Description
pBuffer	IN	array	Pointer for data buffering.
pLength	IN	c_int	The number of bytes expected to be written [OUT] Actual number of bytes written.

◇ Description: Write the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (Only for iRAYPLE Camera). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_ReadUserPrivateData(self, pBuffer, pLength)**

Table 3-13 Parameter description

Parameter	I/O	Type	Description
pBuffer	OUT	array	Pointer for data buffering.
pLength	IN	c_int	The number of bytes expected to be written [OUT] Actual number of bytes written.

◇ Description: Read the user-defined data. User can store the 32,768 bytes of user-defined data inside the camera (Only for iRAYPLE camera). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_WriteUARTData(self, pBuffer, pLength)**

Table 3-14 Parameter description

Parameter	I/O	Type	Description
pBuffer	IN	array	Pointer for data buffering.
pLength	IN	c_int	The number of bytes expected to be written [OUT] Actual number of bytes written.

◇ Description: Write data to the camera serial port register, and history data will be cleared every time user writes (Only for iRAYPLE cameras). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_ReadUARTData(self, pBuffer, pLength)**

Table 3-15 Parameter description

Parameter	I/O	Type	Description
pBuffer	OUT	array	Pointer for data buffering.
pLength	IN	c_int	The number of bytes expected to be written [OUT] Actual number of bytes written.

◇ Description: Read serial port data from the serial register of camera (Only for iRAYPLE camera). If succeeded, return IMV_OK. If failed, return error code.

◇ Reference Routine: None.

3.3 Register Event Callback

- Interface: **IMV_SubscribeConnectArg(self, proc, pUser)**

Table 3-16 Parameter description

Parameter	I/O	Type	Description
proc	IN	IMV_ConnectCallBack	Device connection status event callback function.
pUser	IN	c_void_p	User-defined data, which can be set to None.

◇ Description: Device connection status event callback registration. If succeeded, return **IMV_OK**. If failed, return error code. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: None.

- Interface: **IMV_SubscribeParamUpdateArg(self, proc, pUser)**

Table 3-17 Parameter description

Parameter	I/O	Type	Description
proc	IN	IMV_ParamUpdateCallBack	Parameter update registration event callback function.
pUser	IN	c_void_p	User-defined data, which can be set to None.

  - ◇ Description: Parameter upgrade event callback registration. If succeeded, return **IMV_OK**. If failed, return error code. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.
  - ◇ Reference Routine: None.

- Interface: **IMV_SubscribeStreamArg(self, proc, pUser)**

Table 3-18 Parameter description

Parameter	I/O	Type	Description
proc	IN	IMV_StreamCallBack	Stream channel callback function.
pUser	IN	c_void_p	User-defined data, which can be set to None.

  - ◇ Description: Stream channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.
  - ◇ Reference Routine: None.

- Interface: **IMV_SubscribeMsgChannelArg(self, proc, pUser)**

Table 3-19 Parameter description

Parameter	I/O	Type	Description
proc	IN	IMV_MsgChannelCallBack	Notification channel event callback registration function.
pUser	IN	c_void_p	User-defined data, can be set to None.

  - ◇ Description: Notification channel event callback registration. If succeeded, return **IMV_OK**. If failed, return error code. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.
  - ◇ Reference Routine: None.

3.4 Camera Stream Resource Operation Instruction

3.4.1 Stream Data Configuration

- Interface: **IMV_SetBufferCount(self, nSize)**

Table 3-20 Parameter description

Parameter	I/O	Type	Description
nSize	IN	int	Data buffer quantity.

◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_OK**. If failed, return error code. The configuration cannot be performed during the streaming.

◇ Reference Routine: None.

- Interface: **IMV_ClearFrameBuffer(self)**

◇ Description: Clear the frame data buffer. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GIGE_SetInterPacketTimeout(self, nTimeout)**

Table 3-21 Parameter description

Parameter	I/O	Type	Description
nTimeout	IN	int	Interval time of data packet resending when data packet sending is timeout (unit: ms).

◇ Description: configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. The packet loss resending mechanism under the trigger mode.

◇ Reference Routine: None.

- Interface: **IMV_GIGE_SetSingleResendMaxPacketNum(self, maxPacketNum)**

Table 3-22 Parameter description

Parameter	I/O	Type	Description
maxPacketNum	IN	int	The max number of data packets resending at one time.

◇ Description: Configure the internal time of data packet resending (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. This function is invalid when **maxPacketNum** is 0.

◇ Reference Routine: None.

- Interface: **IMV_GIGE_SetMaxLostPacketNum(self, maxLostPacketNum)**

Table 3-23 Parameter description

Parameter	I/O	Type	Description
maxLostPacketNum	IN	int	The max number of packet loss.

◇ Description: Configure the max number of packet loss of the same frame (only for the GigE model device). If succeeded, return **IMV_OK**. If failed, return error code. This function is invalid when **maxLostPacketNum** is 0. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: None.

- Interface: **IMV_USB_SetUrbTransfer(self, nNum, nSize)**

Table 3-24 Parameter description

Parameter	I/O	Type	Description
nNum	IN	int	Number of data block transmission (Range: 5~256).
nSize	IN	int	The volume of data block transmission (range: 8~512; unit: KByte).

◇ Description: Configure the quantity and volume of data block transmission of the U3V device (only for USB model device). If succeeded, return **IMV_OK**. If failed, return error code.

□ This interface is valid only when the platform is Linux and no driver is adopted, otherwise, it returns the error code of **IMV_NOT_SUPPORT**.

□ Data block transmission quantity: Range: 5~256 (default value: 64).
If this value is too small, it will affect the transmission efficiency; When multiple cameras are connected and used together, this value can be appropriately reduced.

□ The volume of every data block (range: 8~512; default value: 64; unit: KByte).

◇ Reference Routine: None.

3.4.2 Image Collection

- Interface: **IMV_StartGrabbing(self)**

◇ Description: Acquiring images. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

- Interface: **IMV_StartGrabbingEx(self, maxImagesGrabbed, strategy)**

Table 3-25 Parameter description

Parameter	I/O	Type	Description
maxImagesGrabbed	IN	int	The max number of frames of acquired images. After the specified number of images is acquired, the streaming will stop. If the specified number is 0, it means that the streaming is continuously performed.  The parameter of **IMV_StartGrabbing** is 0 by default.
strategy	IN	IMV_EGrabStrategy	Streaming Strategy (**IMV_StartGrabbing** will use the **grabStrartegySequential** to stream).

◇ Description: Acquire images. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_IsGrabbing(self)**

◇ Description: Determine whether the device is streaming. If streaming, return **True**. If not streaming, return **False**.

◇ Reference Routine: None.

- Interface: **IMV_StopGrabbing(self)**

◇ Description: Stop Streaming If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

3.4.3 Acquire ImageData

- Interface: **IMV_AttachGrabbing(self, proc, pUser)**

Table 3-26 Parameter description

Parameter	I/O	Type	Description
proc	IN	IMV_FrameCallBack	Frame data information callback function. Do not handle the time-consuming operations in this function; Otherwise, it will affect the real-time performance of subsequent frame data.
pUser	IN	c_void_p	User-defined data, can be set to None.

◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code.

◇ The asynchronous acquisition mechanism and synchronous acquisition mechanism are mutually exclusive; Therefore, only one of them can be selected in the system for the same device. Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **ChunkData** and **GrabCallback**.

- Interface: **IMV_GetFrame(self, pFrame, timeoutMS)**

Table 3-27 Parameter description

Parameter	I/O	Type	Description
pFrame	OUT	IMV_Frame	Frame data information.
timeoutMS	IN	int	The timeout value for obtaining a frame of an image (unit: ms). The **INFINITE** mode means that it will continuously wait until a frame of an image is received; Otherwise, it will stop streaming.

◇ Description: Receive a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_OK**. If failed, return error code. This interface does not support the multi-thread callings. The synchronous and asynchronous acquisition mechanism (**IMV_AttachGrabbing**) are mutually

exclusive; Therefore, only one of them can be selected in the system for the same device. Acquire images through releasing the internal buffer memory. Use **IMV_ReleaseFrame** to release the image buffer memory.

◇ Reference Routine: **Grab**.

- Interface: **IMV_ReleaseFrame(self, pFrame)**

Table 3-28 Parameter description

Parameter	I/O	Type	Description
pFrame	IN	IMV_Frame	Frame data information.

◇ Description: Release the image buffer memory. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **Grab**.

3.4.4 Operations on Streaming and Image Data

- Interface: **IMV_CloneFrame(self, pFrame, pCloneFrame)**

Table 3-29 Parameter description

Parameter	I/O	Type	Description
pFrame	IN	IMV_Frame	Clone the source frame data information.
pCloneFrame	OUT	IMV_Frame	New frame data information.

◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_OK**. If failed, return error code. Use **IMV_ReleaseFrame** to release the image buffer memory.

◇ Reference Routine: None.

- Interface: **IMV_GetChunkDataByIndex(self, pFrame, index, pChunkDataInfo)**

Table 3-30 Parameter description

Parameter	I/O	Type	Description
pFrame	IN	IMV_Frame	Frame data information.
index	IN	int	Index ID.
pChunkDataInfo	OUT	IMV_ChunkDataInfo	Chunk data information.

◇ Description: Obtain Chunk Data (only for GigE/USB model cameras). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **ChunkData**.

- Interface: **IMV_GetStatisticsInfo(self, pStreamStatsInfo)**

Table 3-31 Parameter description

Parameter	I/O	Type	Description
pStreamStatsInfo	OUT	IMV_StreamStatisticsInfo	Stream statistics information.

◇ Description: Obtain statistics information of streaming (after performing **IMV_StartGrabbing** to call the obtaining command). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_ResetStatisticsInfo(self)**

◇ Description: Reset statistics information of streaming (after performing **IMV_StartGrabbing** to call the resetting command). If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

3.5 Attribute Operation Instruction

3.5.1 Determination, Acquisition and Setting of Attribute

Propriety Determination

- Interface: **IMV_FeatureIsAvailable(self, pFeatureName)**

Table 3-32 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is available. If available, return **True**. If available, return **False**.

◇ Reference Routine: None.

- Interface: **IMV_FeatureIsReadable (self, pFeatureName)**

Table 3-33 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is readable. If readable, return **True**. If unreadable, return **False**.

◇ Reference Routine: None.

- Interface: **IMV_FeatureIsWriteable(self, pFeatureName)**

Table 3-34 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

◇ Reference Routine: None.

- Interface: **IMV_FeatureIsStreamable(self, pFeatureName)**

Table 3-35 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is streamable. If streamable, return **True**. If unstreamable, return **False**.

◇ Reference Routine: None.

- Interface: **IMV_FeatureIsValid(self, pFeatureName)**

Table 3-36 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Determine whether the attribute is valid. If valid, return **True**. If invalid, return **False**.

◇ Reference Routine: **Record**.

Attribute Acquisition

- Interface: **IMV_GetIntFeatureValue(self, pFeatureName, pIntValue)**

Table 3-37 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	Integer attribute value.

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetIntFeatureMin(self, pFeatureName, pIntValue)**

Table 3-38 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

pIntValue	OUT	c_int	The min value that the integer can be configured.

◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetIntFeatureMax(self, pFeatureName, pIntValue)**

Table 3-39 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	The configurable max value of the integer attribute.

◇ Description: Obtain the configurable max value of integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetIntFeatureInc(self, pFeatureName, pIntValue)**

Table 3-40 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pIntValue	OUT	c_int	Integer attribute step size.

◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetDoubleFeatureValue(self, pFeatureName, pDoubleValue)**

Table 3-41 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_double	Floating point attribute value.

◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetDoubleFeatureMin(self, pFeatureName, pDoubleValue)**

Table 3-42 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_double	The configurable min value of

			the floating-point attribute.

◇ Description: Obtain the configurable min value of the floating-point attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetDoubleFeatureMax(self, pFeatureName, pDoubleValue)**

Table 3-43 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pDoubleValue	OUT	c_double	The configurable max value of the floating-point attribute.

◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetBoolFeatureValue(self, pFeatureName, pBoolValue)**

Table 3-44 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pBoolValue	OUT	c_bool	Boolean attribute value.

◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_OK.** If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetStringFeatureValue(self, pFeatureName, pStringValue)**

Table 3-45 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pStringValue	OUT	IMV_String	String attribute value

◇ Description: Obtain the string attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetEnumFeatureValue(self, pFeatureName, pEnumValue)**

Table 3-46 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pEnumValue	OUT	c_int	Enumeration attribute value.

◇ Description: Obtain the enumeration attribute value. If succeeded, return **IMV_OK**. If

failed, return error code.

◇ Reference Routine: **CommPropAccess**.

- Interface: **IMV_GetEnumFeatureSymbol(self, pFeatureName, pEnumSymbol)**

Table 3-47 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pEnumSymbol	OUT	IMV_String	Enumeration symbol value.

◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetEnumFeatureEntryNum(self, pFeatureName, pEntryNum)**

Table 3-48 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pEntryNum	OUT	c_int	The configurable number of the enumeration attribute value.

◇ Description: Obtain the configurable number of the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_GetEnumFeatureEntrys(self, pFeatureName, pEnumEntryList)**

Table 3-49 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pEnumEntryList	OUT	IMV_EnumEntryList	The enumeration value list can be configured in the enumeration attribute value.

◇ Description: Obtain the enumeration value list of the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

Attribute Setting

- Interface: **IMV_SetIntFeatureValue(self, pFeatureName, pIntValue)**

Table 3-50 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

pIntValue	IN	int	The integer attribute value to be configured.

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_SetDoubleFeatureValue(self, pFeatureName, doubleValue)**

Table 3-51 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
doubleValue	IN	double	Floating point attribute value which is to be configured.

◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_SetBoolFeatureValue(self, pFeatureName, boolValue)**

Table 3-52 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
boolValue	IN	bool	Boolean attribute value which is to be configured.

◇ Description: Configure the Boolean attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **ChunkData**.

- Interface: **IMV_SetEnumFeatureValue(self, pFeatureName, enumValue)**

Table 3-53 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
enumValue	IN	int	Enumeration attribute value which is to be configured

◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_SetEnumFeatureSymbol(self, pFeatureName, pEnumSymbol)**

Table 3-54 Parameter description

Parameter	I/O	Type	Description

pFeatureName	IN	str	Attribute name.
pEnumSymbol	IN	str	Enumeration symbol value which is to be configured.

◇ Description: Configure the enumeration symbol value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **ChunkData**.

- Interface: **IMV_SetStringFeatureValue(self, pFeatureName, pStringValue)**

Table 3-55 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.
pStringValue	IN	str	String attribute value which is to be configured.

◇ Description: Configure the string attribute value. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

3.5.2 Execution of Command Attribute

- Interface: **IMV_ExecuteCommandFeature(self, pFeatureName)**

Table 3-56 Parameter description

Parameter	I/O	Type	Description
pFeatureName	IN	str	Attribute name.

◇ Description: Execute the command attribute. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **ChunkData**.

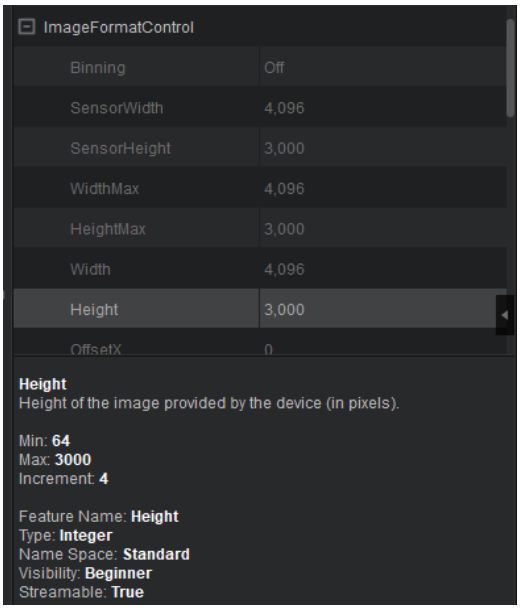
3.5.3 Attribute Search and Usage

User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the Height Attribute. The attribute information of it will be displayed in the below. The data type of the Height Attribute is integer as shown in the figure.

Figure 3-1 Integer attribute of Height



User can obtain and modify the found attributes.

```
heightValue = c_int64(0)

heightMinValue = c_int64(0)

heightMaxValue = c_int64(0)

incrementValue = c_int64(0)


// Obtain attribute value.

ret = cam.IMV_GetIntFeatureValue("Height", heightValue)

if (IMV_OK != ret):

    print("Get feature value failed! ErrorCode[%d]"% ret)

    return ret


// Obtain the configurable min value of attribute. If there is no min value, return the error code.

ret = cam.IMV_GetIntFeatureMin("Height", heightMinValue)

if (IMV_OK != ret):

    print("Get feature minimum value failed! ErrorCode[%d] "% ret)

    return ret


// Obtain the configurable max value of attribute.If there is no max value, return the error code.

ret = cam.IMV_GetIntFeatureMax("Height", heightMaxValue)

if (IMV_OK != ret):

    print("Get feature maximum value failed! ErrorCode[%d]"% ret)
```

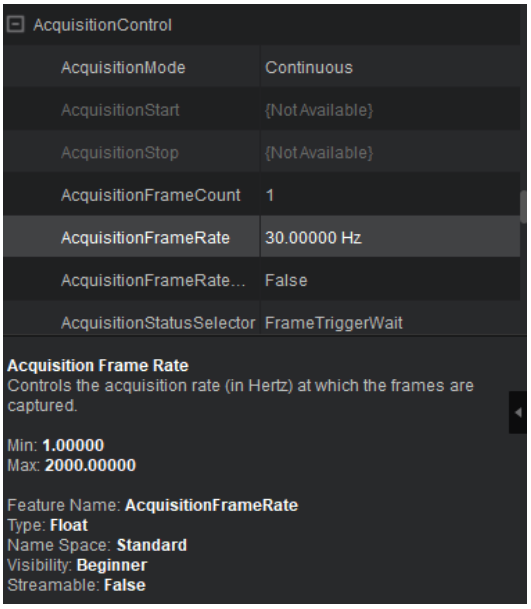
```
        return ret

    // Obtain the attribute step size. If there is no step-size value, return the error code.
    ret = cam.IMV_GetIntFeatureInc("Height", incrementValue)
    if (IMV_OK != ret):
        print("Get feature increment value failed! ErrorCode[%d]"% ret)
        return ret

    // Configure attribute value.
    ret = cam.IMV_SetIntFeatureValue("Height", heightValue.value)
    if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

Floating Point Attribute

Figure 3-2 Floating point attribute




```
    framerate = c_ double(0)
    frameratemin = c_ double(0)
    frameratemax = c_double(0)

    // Obtain attribute value.
    ret = cam.IMV_GetDoubleFeatureValue ("AcquisitionFrameRate", framerate)
    if (IMV_OK != ret):
```

```
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

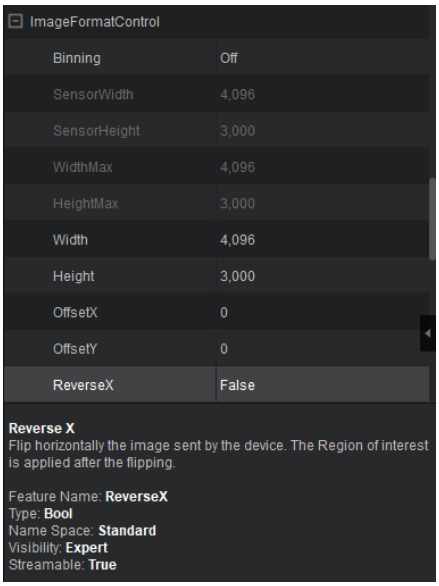
    // Obtain the configurable min value of attribute. If there is no min value, return the error code.
    ret = cam.IMV_GetDoubleFeatureMin("AcquisitionFrameRate", frameratemin)
    if (IMV_OK != ret):
        print("Get feature minimum value failed! ErrorCode[%d]"% ret)
        return ret

    // Obtain the configurable max value of attribute.If there is no max value, return the error code.
    ret = cam.IMV_GetDoubleFeatureMax("AcquisitionFrameRate", frameratemax)
    if (IMV_OK != ret):
        print("Get feature maximum value failed! ErrorCode[%d]"% ret)
        return ret

    // Configure the attribute value.
    ret = cam.IMV_SetDoubleFeatureValue("AcquisitionFrameRate", framerate.value)
    if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

Boolean Attribute

Figure 3-3 Boolean attribute



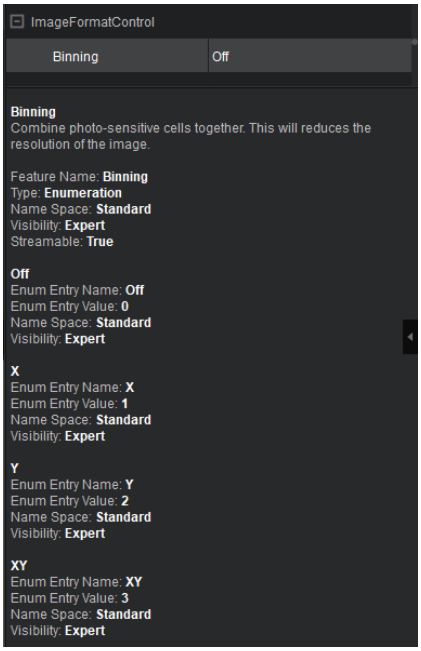

```
    reversevalue = c_bool(0)
```

```
// Obtain the attribute value.
ret = cam.IMV_GetBoolFeatureValue ("ReverseX", reversevalue)
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure the attribute value.
ret = cam.IMV_SetBoolFeatureValue("ReverseX", reversevalue.value)
if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]", ret)
        return ret
```

Enumeration Attribute

Figure 3-4 Enumeration attribute




```
enumValue = c_uint64(0)
pEnumSymbol = IMV_String()
pEntryNum = c_uint(0)
pEnumEntryList = IMV_EnumEntryList()

// Obtain the configurable enumeration value list of attribute.
ret = cam.IMV_GetEnumFeatureValue("Binning", pEnumEntryList)
```

```
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

//Obtain the number of configurable enumeration values of attribute
ret = cam.IMV_GetEnumFeatureEntryNum("Binning", pEntryNum)
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

// Obtain the attribute symbol value.
ret = cam.IMV_GetEnumFeatureSymbol("Binning", pEnumSymbol)
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

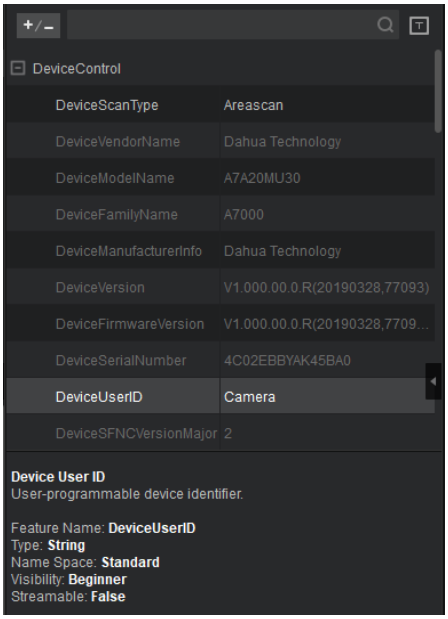
// Obtain the attribute value.
ret = cam.IMV_GetIntFeatureInc("Binning", enumValue)
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure attribute symbol value.
ret = cam.IMV_SetEnumFeatureValue("Binning", pEnumSymbol)
if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure the attribute value.
ret = cam.IMV_SetEnumFeatureValue("Binning", enumValue.value)
if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

String Attribute

Figure 3-5 String attribute



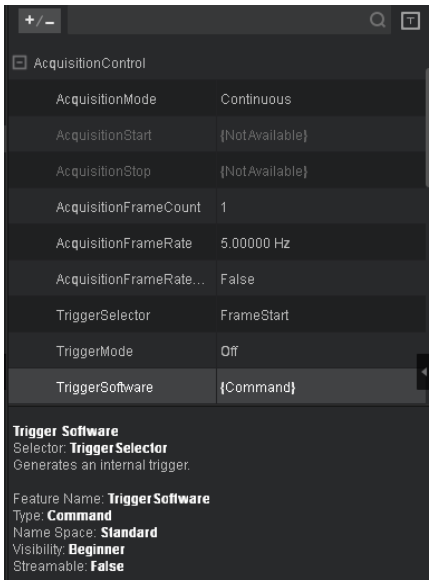

```
id = IMV_String()

// Obtain the attribute value.
ret = cam.IMV_GetStringFeatureValue ("DeviceUserID", id)
if (IMV_OK != ret):
        print("Get feature value failed! ErrorCode[%d]"% ret)
        return ret

// Configure the attribute value.
ret = cam.IMV_SetStringFeatureValue("DeviceUserID", id)
if (IMV_OK != ret):
        print("Set feature value failed! ErrorCode[%d]"% ret)
        return ret
```

Command Attribute

Figure 3-6 Command attribute




```
// Execute the attribute value of command.
ret = cam.IMV_ExecuteCommandFeature ("TriggerSoftware")
if (IMV_OK != ret):
        print("Execute Command feature failed! ErrorCode[%d] " ret)
        return ret
```

3.6 Image Operation

3.6.1 Video Recording

- Interface: **IMV_OpenRecord(self, pstRecordParam)**

Table 3-57 Parameter description

Parameter	I/O	Type	Description
pstRecordParam	IN	IMV_RecordParam	Recording parameter structure.

◇ Description: Start recording. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **Record**.

- Interface: **IMV_InputOneFrame(self, pstRecordFrameInfoParam)**

Table 3-58 Parameter description

Parameter	I/O	Type	Description
pstRecordFrameInfoParam	IN	IMV_RecordFrameInfoParam	Frame information structure for recording.

◇ Description: Record a frame of an image. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: None.

- Interface: **IMV_CloseRecord(self)**

◇ Description: Stop Recording. If succeeded, return **IMV_OK**. If failed, return error code.

◇ Reference Routine: **Record**.

3.6.2 Image Operation

- Interface: **IMV_PixelConvert(self, pstPixelConvertParam)**

Table 3-59 Parameter description

Parameter	I/O	Type	Description
pstPixelConvertParam	IN/OUT	IMV_PixelConvertParam	Pixel format conversion parameter structure.

◇ Description: Pixel format conversion. If succeeded, return **IMV_OK**. If failed, return error code. The supported target pixel format:

gvspPixelRGB8/gvspPixelBGR8/gvspPixelMono8/gvspPixelBGRA8. Convert the raw image data into the pixel format required by the user and stored in the specified memory by the caller.

- When pixel format is **YUV411Packed**, the image width must be divisible by 4.
- When pixel format is **YUV422Packed**, the image width must be divisible by 2.
- When pixel format is **YUYVPacked**, the image width must be divisible by 2.
- Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.
- Reference Routine: **ImageConvert**.

- Interface: **IMV_FlipImage(self, pstFlipImageParam)**

Table 3-60 Parameter description

Parameter	I/O	Type	Description
pstFlipImageParam	IN/OUT	IMV_FlipImageParam	Image flip parameter structure.

◇ Description: Image flip. If succeeded, return **IMV_OK**. If failed, return error code. The supported pixel format of the vertical and horizontal image flip: **gvspPixelRGB8/gvspPixelBGR8/gvspPixelMono8**. Convert the raw image data and store it in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**.

- Interface: **IMV_RotateImage(self, pstRotateImageParam)**

Table 3-61 Parameter description

Parameter	I/O	Type	Description
pstRotateImageParam	IN/OUT	IMV_RotateImageParam	Image flip parameter structure.

◇ Description: Clockwise rotation of the image. If succeeded, return **IMV_OK**. If failed, return error code. The supported image format of the clockwise rotation at 90/180/270 degrees: **gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8**. Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate**.

- Interface: **IMV_ SaveImageToFile(self, pstSaveFileParam)**

Table 3-62 Parameter description

Parameter	I/O	Type	Description
pstSaveFileParam	IN	IMV_SaveImageToFileParam	Image file saving

			parameter structure.

◇ Description: Clockwise rotation of the image. If succeeded, return **IMV_OK**. If failed, return error code. The supported image format includes **BMP, JPEG, PNG** and **TIFF**. The supported max height-width in JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile**.

4 Matched Routine Description

Python Routine Path

【SDK Installation Directory】\Development\ Samples\Python\IMV.

Table 4-1 Routine description

Routine Name	Description
ChunkData	Obtain Block Data.
Grab	Image capturing in free-run mode (active streaming).
GrabCallback	Streaming callback.
ImageConvert	Image format conversion.
ImageFlip	Image flipping.
ImageRotate	Clockwise rotation of the image.
opencv_byCallBack	Display the callback streaming by cv2.
opencv_byGetFrame	Display the active streaming by cv2.
MultiCameraGrab	Multi-camera usage routine.
SaveImageToFile	Image-saving routine.
SoftTrigger	Streaming in software trigger mode.

5 FAQ (Frequently Asked Question)

5.1 Only 8 Images are Obtained by Using the Python Interface

Reason

After acquiring the images using Python interface, the images are not released resulting in the buffer memory being filled and preventing the acquisition of new images.

Solution

Every time the frame is acquired, the buffer memory shall be released by calling the interface of **IMV_ReleaseFrame** as shown in the figure below:

Figure 5-1 IMV_ReleaseFrame calling

```
#get a frame image
nRet=cam.IMV_GetFrame(frame,500)
if IMV_OK!=nRet:
    print("Get frame failed!ErrorCode[%d]" % nRet)
    continue
print("Get frame blockId = [%d]"% frame.frameInfo.blockId)
```

```
#release frame buffer
nRet=cam.IMV_ReleaseFrame(frame)
if IMV_OK!=nRet:
    print("Release frame failed! ErrorCode[%d]" % nRet)
```